



การเขียนโปรแกรมขั้นพื้นฐานด้วยภาษาซี

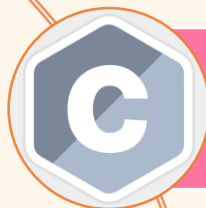
(C Programming Language)

PROGRAMMING
LANGUAGE

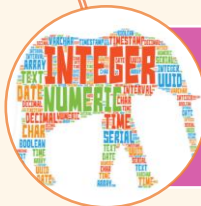
C



AGENDA



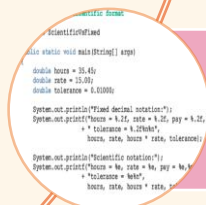
เริ่มต้นภาษาซี



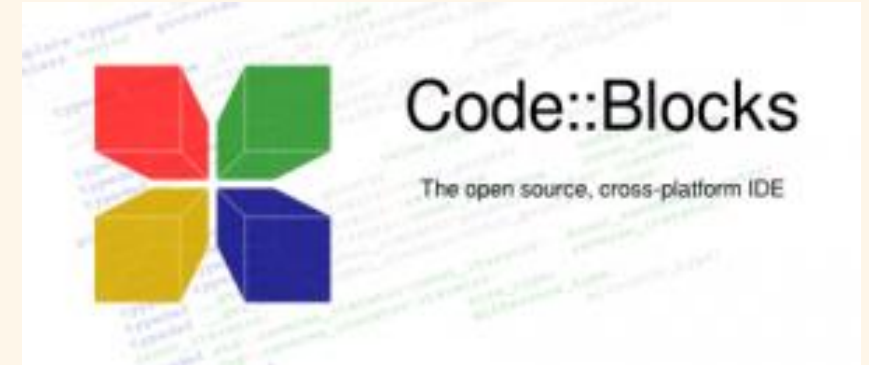
ชนิดของข้อมูล



ตัวดำเนินการ



ฟังก์ชันรับและแสดงผลทางจอภาพ



 **Sublime Text**

1

เป็นภาษาระดับสูง

2

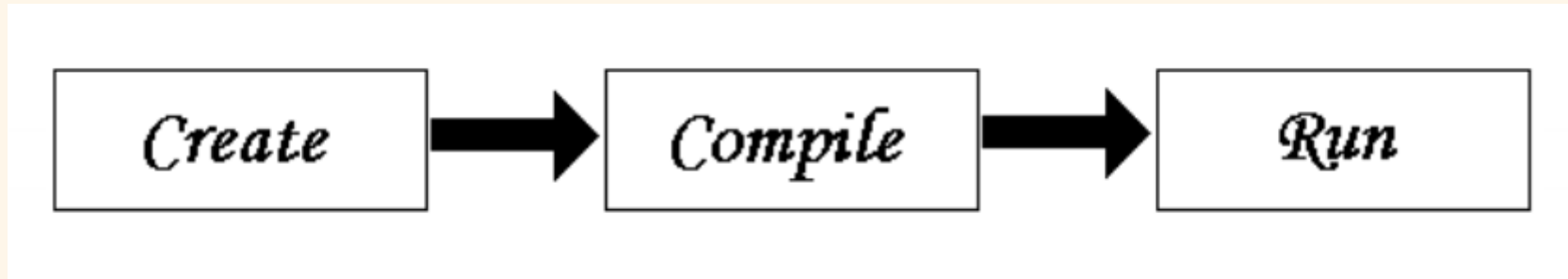
เป็นภาษาโครงสร้าง

3

แทบจะทุกระบบปฏิบัติการมี Complier ภาษาซี

4

มีCompiler รองรับการแปลภาษาซีไปเป็นภาษาเครื่องแทบจะทุกชนิด / ประเภทของเครื่องคอมพิวเตอร์



- 1 การสร้างไฟล์ภาษา นามสกุล (extension) .c
- 2 การคอมไพล์เป็นภาษาเครื่อง
- 3 การรันโปรแกรม



โครงสร้างของโปรแกรมที่เขียนด้วยภาษาซี

Preprocessor Directive

ส่วนหัวของโปรแกรม (Header File) เป็นส่วนสำคัญส่วนหนึ่งของภาษา C เป็นส่วนที่บอกให้คอมไพเลอร์รับทราบว่าให้นำไฟล์ส่วนดังกล่าวมาคอมไพล์

```
1 #include <stdio.h>
```

```
2 main()
```

คำอธิบายภายในโปรแกรม (Program Comment)

```
3 {
```

```
4     printf("Hello Suankularb wittayalai Nonthaburi school "); //Print
```

```
5 }
```

ประโยคคำสั่ง (Compound Statement) เป็นชุดคำสั่งที่บรรจุอยู่ในฟังก์ชันนั้นๆ

ฟังก์ชันหลัก (Main Function) ฟังก์ชัน main() ในภาษา C จัดเป็นฟังก์ชันที่ทำหน้าเสมือนกับเป็นโปรแกรมหลักที่สั่งให้ชุดคำสั่ง ทำงานรวมถึงการเรียกใช้ฟังก์ชันย่อยๆอื่นทำงาน



ไฟล์ .h ที่ตามหลัง include เป็น Header File หรือไฟล์ส่วนหัวที่รวบรวมคำสั่งต่างๆ ของภาษาซีเอาไว้ เพื่อให้ผู้เขียนสามารถใช้ ฟังก์ชันได้ มีอยู่หลายไฟล์เช่น

stdio.h >>> เก็บฟังก์ชันที่ใช้งานทั่วไป เช่น printf , scanf

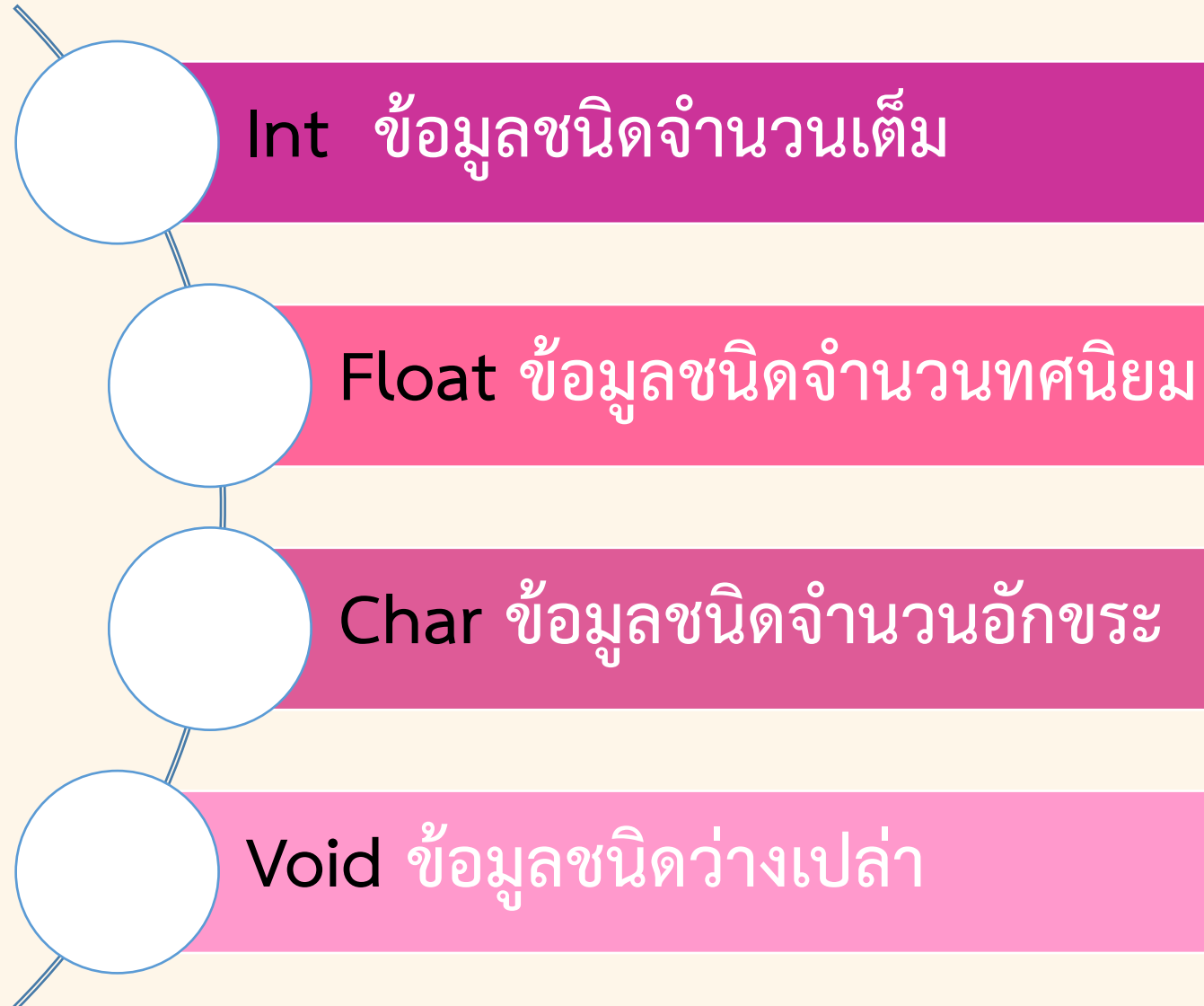
conio.h >>> เก็บฟังก์ชันควบคุมการแสดงผลต่างๆ

string.h >>> เก็บฟังก์ชันที่ใช้ประมวลผลกับข้อความ

math.h >>> เก็บฟังก์ชันที่ใช้ในการคำนวณทางคณิตศาสตร์ เช่น
sin , cos , log



ภาษาซี รู้จักข้อมูล
4 ชนิดหลัก





unsigned int	ข้อมูลชนิดจำนวนเต็มไม่คิดเครื่องหมาย ขนาด 2 หรือ 4 byte
short int	ข้อมูลชนิดจำนวนเต็มขนาด 2 byte
int	ข้อมูลชนิดจำนวนเต็มขนาด 2 หรือ 4 byte (ในคอมพิวเตอร์ 32 bit ตัวแปร int มีขนาด 4 byte แต่ในคอมพิวเตอร์ 16 bit ตัวแปร int มีขนาด 2 byte)
unsigned long	ข้อมูลชนิดจำนวนเต็มไม่คิดเครื่องหมาย 4 byte
long	ข้อมูลชนิดจำนวนเต็ม 4 byte



float	ข้อมูลชนิดจำนวนทศนิยม ขนาด 4 byte
double	ข้อมูลชนิดจำนวนทศนิยม ขนาด 8 byte
long double	ข้อมูลชนิดจำนวนทศนิยม ขนาด 10 byte



char เป็นพารามิเตอร์ ที่ใช้งานเกี่ยวกับ ตัวอักษรและข้อความ ในภาษาซี โดยมี การกำหนดค่าอักขระโดยให้ อยู่ในเครื่องหมาย single quote ('...') เช่น 'C', 'o', 'm', '1' โดยมีรูปแบบการใช้งาน 2 รูปแบบ

unsigned char ข้อมูลชนิดอักขระไม่คิดเครื่องหมาย

char ข้อมูลชนิดอักขระปกติ



Void เป็นพารามิเตอร์ที่ใช้งานในส่วนของฟังก์ชัน

โปรโทไทป์ การสร้างและการใช้งานฟังก์ชัน เพื่อแสดงให้เห็นว่า
ฟังก์ชันที่สร้างขึ้นมาไม่มีการส่งหรือรับค่าจากการเรียกใช้งาน
ฟังก์ชัน



ตารางแสดงขนาดและขอบเขตชนิดของข้อมูล

ชนิดของตัวแปร	ขนาด	ค่าต่ำสุด	ค่าสูงสุด
unsigned char	8 bit	0	255
char	8 bit	-128	127
unsigned int	16 bit	0	65,535
short int	16 bit	-32,768	32,767
int	16 bit	-32,768	32,767
unsigned long	32 bit	0	4,294,967,295
long	32 bit	-2,147,483,648	2,147,483,647
float	32 bit	$\pm 3.4 \times 10^{-38}$	$\pm 3.4 \times 10^{38}$
double	64 bit	$\pm 1.7 \times 10^{-308}$	$\pm 1.7 \times 10^{308}$
long double	80 bit	$\pm 3.4 \times 10^{-4932}$	$\pm 3.4 \times 10^{4932}$



การเขียนโปรแกรมจำเป็นต้องมีการแสดงผล เพื่อให้ผู้ใช้งาน ทราบว่า
โปรแกรมสามารถทำงานอะไร จำเป็นต้องป้อนค่า อะไรบ้าง และเมื่อโปรแกรม
ทำงานเสร็จ ผลลัพธ์ที่ได้เป็นอย่างไร



```
printf (format-string, data-list) ;
```

format-string คือรูปแบบของข้อความซึ่งจะประกอบด้วย
ข้อความธรรมดา, คำรหัส ASCII และ ส่วน
แสดงชนิดข้อมูล โดย format-string จะอยู่ใน
เครื่องหมาย Double quote " "

data-list คือชื่อข้อมูล หรือตัวแปรที่จะทำการแสดงผล
ตามส่วนแสดงชนิดข้อมูลใน format-string

หมายเหตุ คำสั่ง **printf** ต้องเรียกใช้ Preprocessor Directive **#include<stdio.h>**



- ข้อความธรรมดา เป็นส่วนที่แสดงตัวอักษร ตัวเลขออกโดยตรง
- ค่ารหัส ASCII เป็นส่วนที่ควบคุมรูปแบบการแสดงผลเช่น การจัดย่อหน้า การขึ้นต้นบรรทัดใหม่ เป็นต้น
- ส่วนแสดงชนิดข้อมูลเป็นการกำหนดรูปแบบของชนิดข้อมูลที่จะแสดงผลข้อมูล หรือตัวแปร
 - ส่วนแสดงชนิดข้อมูลแบบปกติ
 - ส่วนแสดงชนิดข้อมูลที่กำหนดรูปแบบการแสดงผล



```
#include<stdio.h>
#include<conio.h>
int main()
{
    printf ("Hello World!");
    printf ("This is my first Program.");
    printf ("I am a programmer.");
    return 0;
}
```



ค่ารหัส ASCII สำหรับอักขระพิเศษที่ใช้งานในภาษาซีซึ่งใช้ ควบคุมการแสดงผลในคำสั่ง printf ในส่วนของ format-string

ค่ารหัส ASCII	การใช้งาน
\t	เว้นช่องว่างทุก 1 แท็บ (8 ช่องตัวอักษร)
\n	ขึ้นบรรทัดใหม่
\0	เป็นอักขระว่าง
\'	แสดง single quote ออกทางจอภาพ
\"	แสดง double quote ออกทางจอภาพ
\\	แสดง backslash ออกทางจอภาพ



ตัวอย่างการใช้งาน printf โดยใช้ค่ารหัส ASCII

```
#include<stdio.h>
int main()
{
    printf ("Hello World!\n");
    printf ("This is my first Program.");
    printf ("\n\tI\'m a programmer.");
    return 0;
}
```

ในการเขียนโปรแกรมที่มีการประยุกต์ขึ้น เช่น โปรแกรมคำนวณเลข จะต้องมีการแสดงผลลัพธ์ของค่าที่ทำการคำนวณ ซึ่งคำสั่ง `printf` สามารถแสดงผลค่าตัวแปรได้ แต่จะต้องเขียนโปรแกรมให้มีความสัมพันธ์กันในส่วนของ `format-string` และ `data-list`

```
printf (format-string, data-list);
```

- โดยที่ใน `format-string` จะมีส่วนแสดงชนิดข้อมูล
- และใน `data-list` จะมีข้อมูล หรือตัวแปรที่จะแสดงผล

ส่วนแสดงชนิดข้อมูล	การใช้งาน
%d	แสดงผลข้อมูลชนิดจำนวนเต็ม
%u	แสดงผลข้อมูลชนิดจำนวนเต็มบวก (ไม่คิดเครื่องหมาย)
%o	แสดงผลข้อมูลเป็นเลขฐานแปด
%x	แสดงผลข้อมูลเป็นเลขฐานสิบหก
%f	แสดงผลข้อมูลชนิดจำนวนทศนิยม (6 ตำแหน่ง)
%e	แสดงผลข้อมูลเป็นจำนวนทศนิยมและอยู่ในรูปยกกำลัง
%c	แสดงผลข้อมูลชนิดอักขระ
%s	แสดงผลข้อมูลชนิดข้อความ
%p	แสดงผลข้อมูลชนิดตัวชี้ตำแหน่ง



```
#include<stdio.h>
int main()
{
    printf ("My name is : %s\n", "Kmitl");
    printf ("My point : %d\n", 10+40+49);
    printf ("Grade : %c\n", 'A');
    printf ("GPA : %f", 3.99);
    return 0;
}
```



- การแสดงผลตัวแปรโดยใช้คำสั่ง printf สามารถใช้งานโดยการใส่ข้อมูลหรือชื่อตัวแปรในส่วนของ data-list
- หากไม่ต้องการแสดงผลตัวแปรไม่ต้องมีส่วนของ data-list
- หากต้องการแสดงผลข้อมูลหรือตัวแปร มากกว่าหนึ่งตัวในคำสั่งให้ใช้เครื่องหมาย comma , สำหรับคั่นชื่อตัวแปร โดยจะทำการแสดงผลตามลำดับแปรและส่วนการแสดงผลชนิดข้อมูล



การกำหนดรายชื่อตัวแปรในคำสั่ง printf

```
printf ("...%?... ", data);
```

...data...

ตัวอย่าง

```
printf ("...%d...", 19);
```

...19...

```
printf ("%? %? ... %?", data-1, data-2, ..., data-n);
```

data-1 data-2 ... data-n

ตัวอย่าง

```
printf ("%d-%d-%d", 19, 1, 1980);
```

19-1-1980



การกำหนดรายชื่อตัวแปรในคำสั่ง printf

```
#include<stdio.h>
int main()
{
    printf ("Age = %d, GPA = %f\n",17,3.75) ;
    printf ("Programming: %f\nMechanics: %f",4.0,3.5) ;
    return 0;
}
```

```
#include<stdio.h>
int main()
{
    printf ("Subject : %s(%d)\n", "Programming",2552) ;
    printf ("Point      : %d\nGrade      : %c",99,'A') ;
    return 0;
}
```



```
printf ("%m?", data) ;
```

- เป็นการจองพื้นที่หน้าจอจำนวน m ตัวอักษร แล้วแสดงผล data จัดด้านขวาของพื้นที่ส่วนที่จองไว้ โดยชนิดข้อมูลตาม ? (หากความยาวเกินส่วนที่จองไว้ก็จะเลื่อนออกไป)

```
printf ("%~m?", data) ;
```

- เป็นการจองพื้นที่หน้าจอขนาด m ตัวอักษร แล้วแสดงผล data จัดด้านซ้ายของพื้นที่ส่วนที่จองไว้ โดยชนิดข้อมูลตาม ?



ตัวอย่างการใช้ printf โดยมีการกำหนดรูปแบบ

```
#include<stdio.h>
int main()
{
    printf ("123456789012345678901234567890");
    printf ("\n%20d*",46);
    printf ("\n%-20d*",46);
    printf ("\n%3d*",46);
    printf ("\n%3d*",2550);
    return 0;
}
```

```
123456789012345678901234567890
                                46*
46                                *
    46*
2550*
```



ตัวอย่างการใช้ printf โดยมีการกำหนดรูปแบบ

```
#include <stdio.h>
int main()
{
    printf ("123456789012345678901234567890");
    printf ("\n%20c*", 'c');
    printf ("\n%-20c*", 'c');
    printf ("\n%10s*", "Pro");
    printf ("\n%10s*", "Programming");
    return 0;
}
```

```
123456789012345678901234567890
                                c*
c                                *
        Pro*
Programming*
```



```
printf ("% .n?", data) ;
```

- เป็นการกำหนดให้แสดงจำนวนทศนิยม n ตำแหน่ง สำหรับ %f หรือแสดงอักขระจำนวน n ตัว สำหรับ %s

```
printf ("%m.n?", data) ;
```

- เป็นการจองพื้นที่หน้าจอขนาด m ตัวอักษรแล้วแสดงผล data จำนวนทศนิยม n ตำแหน่ง สำหรับ %f หรือแสดงอักขระจำนวน n ตัว สำหรับ %s



ตัวอย่างการใช้ printf โดยมีการกำหนดรูปแบบ

```
printf ("123456789012345678901234567890");  
printf ("\n%20s*", "programming");  
printf ("\n%-20s*", "programming");  
printf ("\n%.3s*", "programming");  
printf ("\n%20.3s*", "programming");  
printf ("\n%-20.3s*", "programming");
```

```
123456789012345678901234567890
```

```
        programming*
```

```
programming          *
```

```
pro*
```

```
                pro*
```

```
pro                  *_
```



ตัวอย่างการใช้ printf โดยมีการกำหนดรูปแบบ

```
printf ("123456789012345678901234567890") ;  
printf ("\n%20f*", 1234.56789) ;  
printf ("\n%-20f*", 1234.56789) ;  
printf ("\n%.3f*", 1234.56789) ;  
printf ("\n%20.3f*", 1234.56789) ;  
printf ("\n%-20.3f*", 1234.56789) ;
```

123456789012345678901234567890

1234.567890*

1234.567890 *

1234.568*

1234.568*

1234.568 * _



ตัวแปร (Variable) คือ การจองพื้นที่ในหน่วยความจำของคอมพิวเตอร์สำหรับเก็บข้อมูลที่ต้องใช้ในการทำงานของโปรแกรม โดยมีการตั้งชื่อเรียกหน่วยความจำในตำแหน่งนั้นด้วย เพื่อความสะดวกในการเรียกใช้ข้อมูล ถ้าจะใช้ข้อมูลใดก็ให้เรียกผ่านชื่อของตัวแปรที่เก็บเอาไว้



การเขียนคำสั่งแบบใช้ตัวแปร : ชื่อของตัวแปร

ในการประกาศสร้างตัวแปรต้องมีการกำหนดชื่อ ซึ่งชื่อนั้นไม่ใช่ว่าจะตั้งให้สื่อความหมายถึงข้อมูลที่เก็บอย่างเดียวโดยไม่คำนึงถึงอย่างอื่น เนื่องจากภาษา C มีข้อกำหนดในการตั้งชื่อตัวแปรเอาไว้ แล้วถ้าตั้งชื่อผิดหลักการเหล่านี้ โปรแกรมจะไม่สามารถทำงานได้ หลักการตั้งชื่อตัวแปรในภาษา C แสดงไว้ดังนี้

ต้องขึ้นต้นด้วยตัวอักษร A-Z หรือ a-z หรือเครื่องหมาย _ (Underscore) เท่านั้น

ภายในชื่อตัวแปรสามารถใช้ตัวอักษร A-Z หรือ a-z หรือตัวเลข 0-9 หรือเครื่องหมาย _

ภายในชื่อห้ามเว้นช่องว่าง หรือใช้สัญลักษณ์นอกเหนือจากข้อ 2

ตัวอักษรเล็กหรือใหญ่มีความหมายแตกต่างกัน

ความยาวของตัวแปรไม่เกิน 31 ตัวอักษร (เกินได้แต่โปรแกรมจะคำนวณ 31 ตัวแรก)

ห้ามตั้งชื่อซ้ำกับคำสงวน (Reserved Word)



การเขียนคำสั่งแบบใช้ตัวแปร : Reserved Word

auto	default	float	register	struct	volatile	break	do
far	return	switch	while	case	double	goto	short
typedef	char	else	if	signed	union	const	enum
int	sizeof	unsigned	continue	extern	long	static	void



การเขียนคำสั่งแบบใช้ตัวแปร : ตัวอย่างการตั้งชื่อที่ถูกต้องและไม่ถูกต้อง

bath_room	ถูกต้อง
n-sync	ผิดหลักการ เนื่องจากมีเครื่องหมาย - ปรากฏในชื่อ
108dots	ผิดหลักการ เนื่องจากขึ้นต้นด้วยตัวเลข
Year#	ผิดหลักการ เนื่องจากมีเครื่องหมาย # อยู่ในชื่อ
_good	ถูกต้อง
goto	ผิดหลักการ เนื่องจากเป็นคำสงวน
work	ถูกต้อง
break	ผิดหลักการ เนื่องจากเป็นคำสงวน



การเขียนคำสั่งแบบใช้ตัวแปร : ตัวอย่างการประกาศตัวแปร

```
int    number;
```

```
int    a, b, c;
```

```
float  real;
```

```
float  point1, point2;
```

```
char   choice;
```

```
char   ch1, ch2;
```



การเขียนคำสั่งแบบใช้ตัวแปร : รูปแบบการประกาศตัวแปรในภาษา C

```
type var1 = value1;
```

```
type var1 = value1, varN = valueN;
```

type คือ ชนิดของข้อมูลที่จะกำหนดให้กับตัวแปร

varX คือ ชื่อของตัวแปรที่จะตั้ง

valueX คือ ค่าของตัวแปรที่ต้องการกำหนดให้



```
int    number = 25;
```

```
int    a = 1, b = 2, c = 3;
```

```
float  real = 99.99;
```

```
float  point1 = 45.2, point2 = 30;
```

```
char   choice = 'a';
```

```
char   ch1 = 'o', ch2 = 'z';
```



ในภาษาซีจะไม่มีข้อมูลชนิดข้อความโดยเฉพาะ ซึ่งในการเขียนโปรแกรมส่วนใหญ่จำเป็นต้องรับค่าข้อมูล ที่เป็นข้อความ เราสามารถใช้ตัวแปรข้อมูลชนิดอักขระหลายๆ ตัวมาใช้งานได้ในระดับหนึ่งแต่ก็ยังไม่สะดวก เมื่อข้อความมีความยาวมาก ตัวอย่างเช่น ต้องการใช้ข้อความว่า Hello สามารถใช้ตัวแปรชนิดอักขระ5 ตัวแทน

```
charch1='H',ch2='e',ch3='l', ch4='l',ch5='o';
```

- ตัวแปรชนิดข้อความในภาษาซี คือ การนำอักขรมาเรียนต่อกัน ดังนั้นสามารถสร้างตัวแปรชนิดอักขระเรียงต่อกันหลาย ๆ ตัว ให้เป็นตัวแปรชนิดแถวลำดับ ทำให้สามารถใช้เก็บข้อมูลชนิดข้อความได้
- โดยตัวแปรชนิดข้อความในภาษาซีจะอยู่ในเครื่องหมาย Double quote " "



```
char    var1[M1] ;
```

```
char    var1[M1] , var2[M2] ;
```

varX คือ ชื่อตัวแปร

MX คือ จำนวนของอักขระที่จะใช้เก็บบวกด้วย 1

การใช้ตัวแปรแถวลำดับชนิดอักขระเป็นตัวแปรข้อความ ในภาษาซีกำหนดไว้ว่าตัวสุดท้ายของตัวแปรแถวลำดับคือ \0

\0 → Null character



```
char    var[M] = "??...?";
```

```
char    var[M] = {'?', '?', ..., '?'};
```

```
char    var[] = "??...? ";
```

var คือ ชื่อตัวแปร

M คือ จำนวนของอักขระที่จะใช้เก็บบวกด้วย 1

? คือ อักขระที่จะกำหนดค่าให้ข้อความ จำนวน $m-1$ ตัว



ตัวอย่างการประกาศตัวแปรและกำหนดค่า

```
char subject[12] = "Programming";  
char nick[4] = "Com";  
char nick_1[4] = {'C', 'o', 'm', '\0'};  
char name[] = "Somsak";
```

subject[12]

P	r	o	g	r	a	m	m	i	n	g	\0
---	---	---	---	---	---	---	---	---	---	---	----

nick[4]

c	o	m	\0
---	---	---	----

name[]

S	o	m	s	a	k	\0
---	---	---	---	---	---	----

nick_1[4]

C	o	m	\0
---	---	---	----



variable[**N**]

variable คือชื่อตัวแปร

N คือลำดับอักษรที่จะอ้างอิงในตัวแปรข้อความ

เริ่มนับอักษรตัวแรกเป็นตำแหน่งที่ 0



```
char    subject[12] = "Programming";
```

subject[12]

P	r	o	g	r	a	m	m	i	n	g	\0
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]

subject[0] → 'P'

subject[1] → 'r'

subject[2] → 'o'

subject[10] → 'g'



ตัวอย่างการใช้งานตัวแปรร่วมกับคำสั่ง printf

- ในคำสั่ง printf มีส่วนแสดงชนิดข้อมูล ซึ่งตัวแปรที่ใช้งานก็เป็นส่วนย่อยของข้อมูลชนิดต่าง ๆ

```
printf (format-string, data-list);
```

- จึงสามารถใช้ตัวแปรแทนในส่วนของ data-list ได้ เช่น

```
#include<stdio.h>
int main()
{
    printf ("GPA : %.2f", 3.5);
    return 0;
}
```

```
#include<stdio.h>
int main()
{
    float g = 3.5;
    printf ("GPA : %.2f", g);
    return 0;
}
```



ตัวอย่างการใช้คำสั่ง printf โดยไม่ใช้ตัวแปร

```
#include <stdio.h>

] main(){
    printf("%s\n", "suankularb wittayalai nonthaburi school");
    printf("%s\n", "suankularb wittayalai nonthaburi school");
    printf("%s\n", "suankularb wittayalai nonthaburi school");
    printf("%s\n", "suankularb wittayalai nonthaburi school");
    printf("%s\n", "suankularb wittayalai nonthaburi school");
    printf("%s\n", "suankularb wittayalai nonthaburi school");
    printf("%s\n", "suankularb wittayalai nonthaburi school");
    printf("%s\n", "suankularb wittayalai nonthaburi school");
```

หากต้องการเปลี่ยนเป็น
Hello ทั้งหมด ทำอย่างไร??



ตัวอย่างการใช้คำสั่ง printf โดยไม่ใช่ตัวแปร

```
#include <stdio.h>

main(){
    char name[10] = "Hello";

    printf("%s\n", name);
    printf("%s\n", name);
    printf("%s\n", name);
    printf("%s\n", name);
    printf("%s\n", name);
    printf("%s\n", name);
    printf("%s\n", name);
    printf("%s\n", name);
}
```

หากต้องการเปลี่ยนเป็น
ข้อความให้เป็นไปตามที่ผู้ใช้กำหนด
ทำอย่างไร??



- โปรแกรมโดยทั่วไปต้องมีการรับค่าข้อมูล เพื่อนำมาหาผลลัพธ์ตามกระบวนการทำงานของโปรแกรม หรือ ตามความต้องการของผู้ใช้งาน
- คำสั่งที่ใช้รับค่าข้อมูลในภาษาซีมีหลายคำสั่ง แต่ที่สามารถใช้งานได้ครอบคลุมและนิยมนำมาใช้กันคือคำสั่ง scanf



scanf (*format-string*, *address-list*) ;

format-string คล้ายในคำสั่ง printf แต่จะมีเฉพาะส่วนแสดงชนิดข้อมูล และอยู่ในเครื่องหมาย " "

address-list คือ ตำแหน่งของตัวแปรที่ต้องการเก็บข้อมูลไว้ (การใช้งานตำแหน่งของตัวแปรจะใช้เครื่องหมาย & นำหน้าชื่อตัวแปร ยกเว้นตัวแปรชนิดข้อความ)

หมายเหตุ คำสั่ง scanf ต้องเรียกใช้ Preprocessor Directive `#include<stdio.h>`



ตัวอย่างการใช้คำสั่ง scanf

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    float point;
```

```
    char  name[20];
```

```
    printf ("Enter your name  : ");
```

```
    scanf  ("%s",name);
```

```
    printf ("Enter your point : ");
```

```
    scanf  ("%f",&point);
```

```
    return 0;
```

```
}
```

ตัวแปรข้อความ

ไม่ต้องมี & หน้าตัวแปร

ตัวแปรทศนิยม

ต้องมี & หน้าตัวแปร



ตัวอย่างการใช้คำสั่งprintf ร่วมกับ scanf

```
1  #include<stdio.h>
2
3  int main(){
4      char first[20],last[20];
5      printf ("Enter your name and surname : ");
6      scanf ("%s %s",first,last);
7      printf ("Hi %s %s\nHow are you?",first,last);
8      return 0;
9  }
```



C:\Users\watchara\Desktop\d1\ohayoutest

```
Enter your name and surname : Watchara Ananpitakgoon
Hi Watchara Ananpitakgoon
How are you?
-----
```



- คำสั่ง scanf ไม่สามารถใช้รับข้อความที่มีการเว้นเพื่อเก็บในตัวแปรข้อความตัวเดียวได้ (เมื่อใช้ %s)
- เราสามารถใช้คำสั่ง scanf เพื่อให้รับข้อความที่มีการเว้นไปเก็บในตัวแปรชนิดข้อความได้ดังนี้

```
scanf ("%[^\\n]", string) ;
```



ตัวอย่างการใช้ scanf รับข้อความที่มีการเว้น (spacebar)

```
#include<stdio.h>
int main()
{
    char  name[40];
    float gpa;
    printf ("Enter your name : ");
    scanf ("%[^\\n]",name);
    printf ("Enter your GPA  : ");
    scanf ("%f",&gpa);
    printf ("Name : %s\\n",name);
    printf ("Gpa  : %f",gpa);
    return 0;
}
```

(Inactive C:\TCWIN45\BIN\NONAME00.EXE)

Enter your name : Com eng

Enter your GPA : 3.7

Name : Com eng

Gpa : 3.700000





เครื่องหมายการคำนวณทางคณิตศาสตร์

เครื่องหมาย	การทำงาน	ตัวอย่าง
+	บวก	<code>ans = a + b;</code>
-	ลบ	<code>ans = a - b;</code>
*	คูณ	<code>ans = a * b;</code>
/	หาร	<code>ans = a / b;</code>
%	โมดูลัส (modulo)	<code>ans = a % b;</code>

- การกำหนดค่าให้กับตัวแปรในภาษาซีใช้เครื่องหมาย = โดยการ
ทำงานจะนำค่าที่อยู่ทางขวามือ (จำนวน อักขระ ข้อความ ค่าจาก
ตัวแปร หรือผลลัพธ์จากฟังก์ชัน) ให้กับตัวแปรที่อยู่ทางซ้ายมือ

```
variable = value;
```

```
num1 = 99;
```

```
n = num % 10;
```

```
point = mid + final; ch = '9';
```

```
ans = pow(x,y);
```



เครื่องหมาย	การทำงาน	ตัวอย่าง	ขั้นตอนการทำงาน	
++	เพิ่มค่าทีละ 1 (Increment)	$x++$; $++x$;	เพิ่มค่า x ขึ้น 1	
		$y = ++x$;	เพิ่มค่า x ขึ้น 1	กำหนดค่าให้ y
		$y = x++$;	กำหนดค่าให้ y	เพิ่มค่า x ขึ้น 1
--	ลดค่าทีละ 1 (Decrement)	$x--$; $--x$;	ลดค่า x ลง 1	
		$y = --x$;	ลดค่า x ลง 1	กำหนดค่าให้ y
		$y = x--$;	กำหนดค่าให้ y	ลดค่า x ลง 1



เครื่องหมาย	การทำงาน	ตัวอย่าง	ขั้นตอนการทำงาน	
++	เพิ่มค่าทีละ 1 (Increment)	$x++$; $++x$;	เพิ่มค่า x ขึ้น 1	
		$y = ++x$;	เพิ่มค่า x ขึ้น 1	กำหนดค่าให้ y
		$y = x++$;	กำหนดค่าให้ y	เพิ่มค่า x ขึ้น 1
--	ลดค่าทีละ 1 (Decrement)	$x--$; $--x$;	ลดค่า x ลง 1	
		$y = --x$;	ลดค่า x ลง 1	กำหนดค่าให้ y
		$y = x--$;	กำหนดค่าให้ y	ลดค่า x ลง 1



ตัวดำเนินการ && ตัวถูกดำเนินการ

Type 1	Operator	Type 2	Result	Exam	
int	+, -, *, /	int	int	3*3	9
				19/2	9
int	+, -, *, /	float	float	3*3.0	9.000000
				19/2.0	9.500000
float	+, -, *, /	int	float	3.0*3	9.000000
				19.0/2	9.500000
float	+, -, *, /	float	float	3.0*3.0	9.000000
				19.0/2.0	9.500000
int	%	int	int	7%4	3
				8%4	0

ลำดับความสำคัญ	เครื่องหมาย
1	()
2	!, ++, --, (type)
3	*, /, %
4	+, -
5	<, <=, >, >=
6	==, !=
7	&&
8	
9	*=, /=, %=, +=, -=



ตัวอย่างลำดับแสดงความสำคัญในภาษาซี

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a = 10;
```

```
    printf("%d", 10*2*3-10/7);
```

```
    printf("%d", 10*2*(3-10)/7);
```

```
    printf("%d", 10*2>2+3);
```

```
    printf("%d", 10*2<2+3);
```

```
    return 0;
```

```
}
```

//ผลลัพธ์

59 -20 1 0



```
putchar (ch) ;
```

```
puts (str) ;
```

```
ch = getchar() ;
```

```
ch = getch() ;
```

```
gets (str) ;
```



- เป็นคำสั่งที่ใช้รับข้อมูลชนิดอักขระจากผู้ใช้งานเพียงตัวเดียว โดยเมื่อป้อนข้อมูลแล้วต้องกด Enter
- คำสั่ง getchar มีรูปแบบการใช้งานคำสั่งดังนี้

```
ch = getchar() ;
```

ch คือตัวแปรชนิดข้อความที่ต้องการเก็บข้อมูลไว้

หมายเหตุ คำสั่ง getchar ต้องเรียกใช้ Preprocessor Directive `#include<stdio.h>`



- เป็นคำสั่งที่ใช้รับข้อมูลชนิดอักขระจากผู้ใช้งานเพียงตัวเดียว โดยเมื่อป้อนข้อมูลแล้วโปรแกรมจะทำงานคำสั่งต่อไปทันที และจะไม่แสดงอักขระที่พิมพ์ไป
- คำสั่ง getch มีรูปแบบการใช้งานคำสั่งดังนี้

```
ch = getch ( ) ;
```

ch คือตัวแปรชนิดข้อความที่ต้องการเก็บข้อมูลไว้

หมายเหตุ คำสั่ง getch ต้องเรียกใช้ Preprocessor Directive `#include<conio.h>`



- เป็นคำสั่งที่ใช้รับข้อมูลชนิดข้อความจากผู้ใช้งาน โดยสามารถใส่ข้อมูลที่มีการเว้นช่องว่างภายในได้
- คำสั่ง gets มีรูปแบบการใช้งานคำสั่งดังนี้

```
gets (str) ;
```

str คือตัวแปรชนิดข้อความที่ต้องการแสดงผล

หมายเหตุ คำสั่ง gets ต้องเรียกใช้ Preprocessor Directive `#include<stdio.h>`



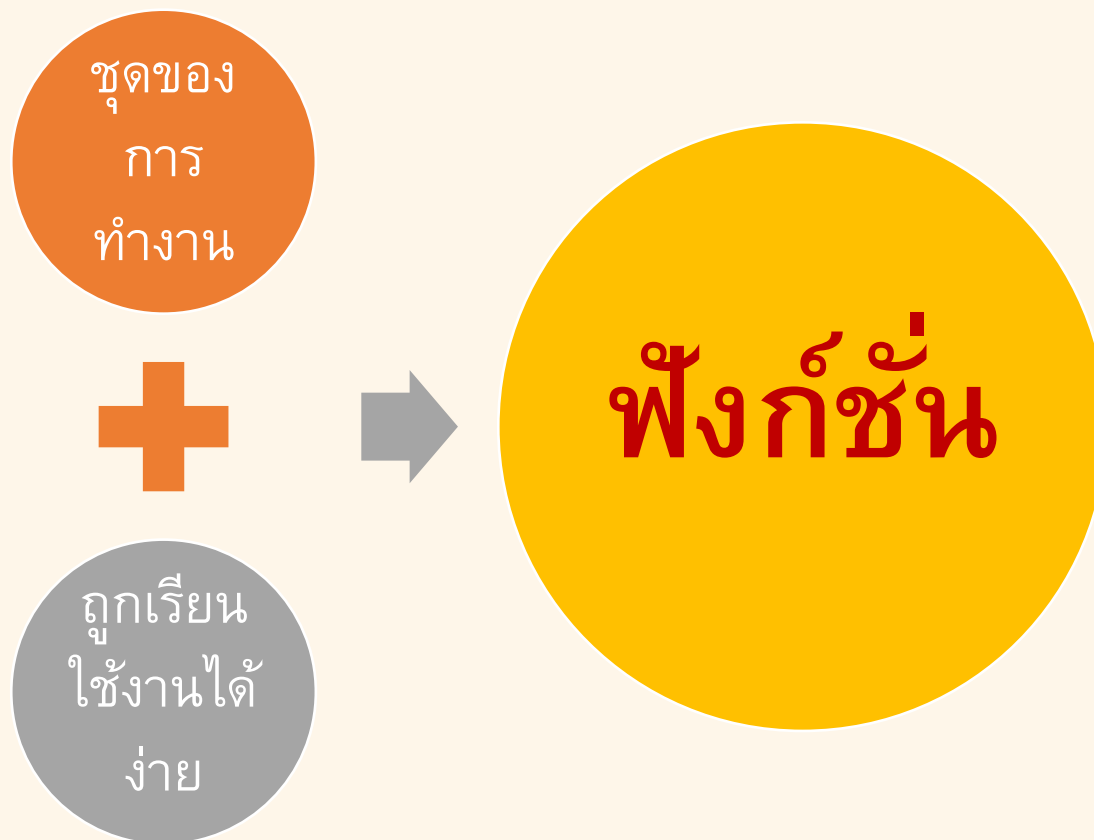
```
char ch1,ch2;  
printf ("Enter Character 1 : ");  
ch1 = getchar();  
printf ("Enter Character 2 : ");  
ch2 = getch();  
puts ("\n**** Output ****");  
printf ("Char 1 = %c\nChar 2 = %c", ch1,ch2);
```

```
Enter Character 1 : J  
Enter Character 2 :  
**** Output ****  
Char 1 = J  
Char 2 = 0
```

พิมพ์ O แต่ไม่แสดงออกมา



ฟังก์ชัน คือ ชุดของการทำงาน ที่ถูกเขียนขึ้นให้โปรแกรมเมอร์เรียกใช้งานได้ง่าย



ฟังก์ชันแบ่งได้ 2 ประเภท

ฟังก์ชันไลบรารีมาตรฐาน (Standard Library Function)

ฟังก์ชันที่สร้างขึ้นเอง (User Defined Function)



ฟังก์ชันไลบรารีมาตรฐาน (Standard Library Function)

- ฟังก์ชันไลบรารีมาตรฐาน (Standard Library Function) เป็นฟังก์ชันที่มีอยู่แล้วใน Library ในการใช้ต้อง Include Directive ก่อน
- Directive คือ สารบัญของกลุ่มฟังก์ชัน เช่น `stdio.h` , `conio.h` , `string.h` , `math.h`;
- การ Include Directive จะเป็นเหมือนการประกาศให้กับ Compile ทราบว่าจะใช้คำสั่งในกลุ่มของ Directive นั้นๆ เช่นการใช้คำสั่ง `sin()` ซึ่งอยู่ใน `math.h` จะต้องมีการ Include `math.h` เสมอ ดังตัวอย่าง



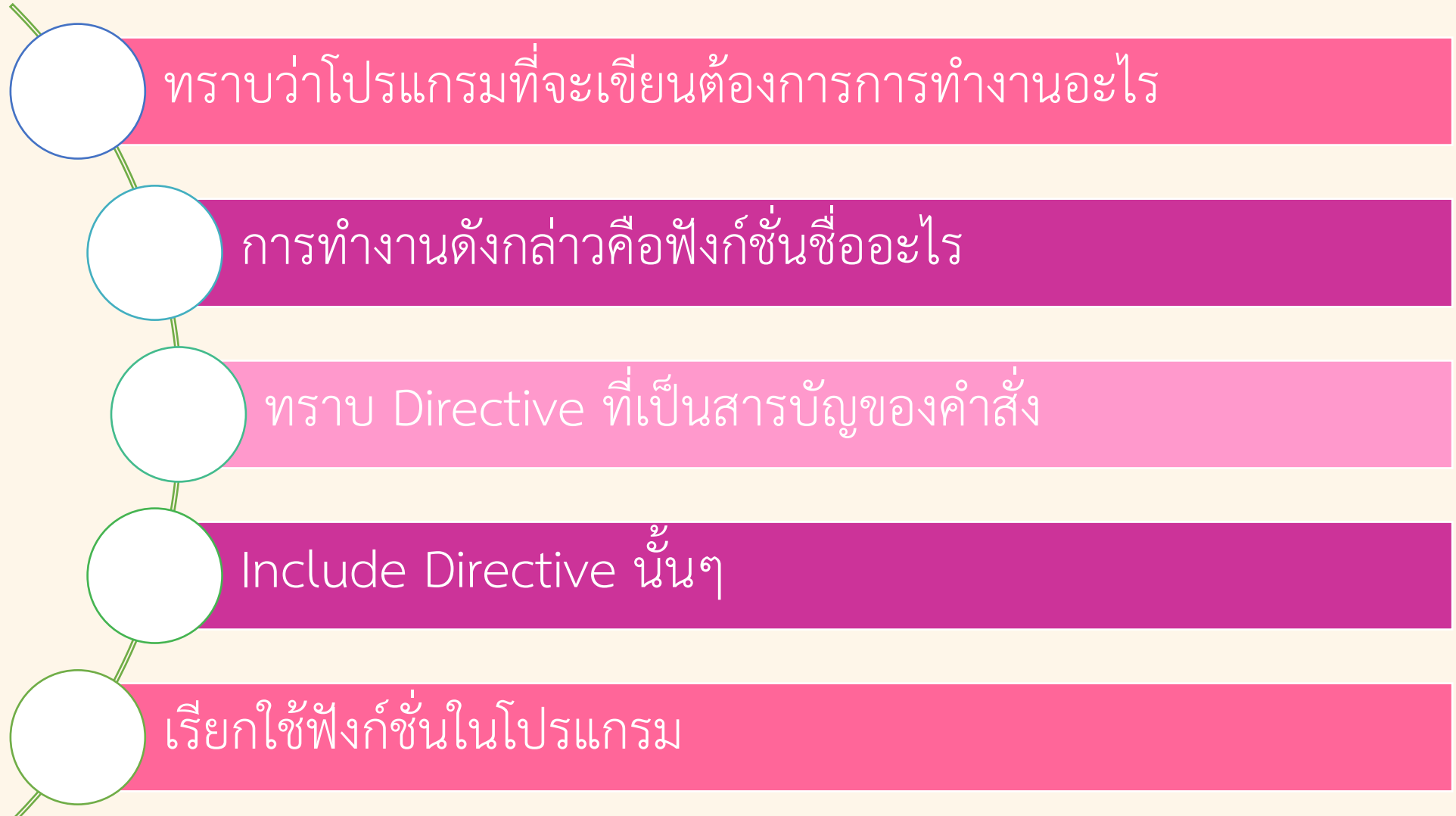
ฟังก์ชันไลบรารีมาตรฐาน (Standard Library Function)

```
#include<stdio.h>
#include<conio.h>
#include<math.h>
int main()
{
    double rad = -1.0;
    do {
        printf ( "Sine of %f is %f\n", rad, sin(rad) );
        rad += 0.1;
    } while (rad <= 1.0);
    return 0;
}
```



ฟังก์ชันไลบรารีมาตรฐาน (Standard Library Function)

การเรียกใช้ฟังก์ชันไลบรารีมาตรฐาน (Standard Library Function)





ฟังก์ชันการคำนวณทางคณิตศาสตร์

- ไฟล์ header >> math.h

ฟังก์ชันสำหรับอักขระและความ

- ไฟล์ header >> string.h
ctype.h



```
#include<math.h>
```

```
sin(var) ;
```

```
cos(var) ;
```

```
tan(var) ;
```

```
sqrt(var) ; //รากที่ 2
```

```
pow(var1,var2) ; //var1 ยกกำลัง var2
```

```
log(var) ; // log ฐาน e
```

```
log10(var) ; // log ฐาน 10
```

```
exp(var) ; // e ยกกำลัง
```

```
fabs(var) ; // ค่าสัมบูรณ์แบบfloat
```



```
#include<stdio.h>
#include<conio.h>
#include<math.h>
#define PI 3.14
int main()
{
    float deg,rad;
    printf ("Enter Degree : ");
    scanf ("%f",&deg);
    rad = deg * PI / 180;
    printf ("sin(%.2f) = %.3f\n",deg,sin(rad));
    printf ("cos(%.2f) = %.3f\n",deg,cos(rad));
    printf ("tan(%.2f) = %.3f\n",deg,tan(rad));
    return 0;
}
```



```
#include<string.h>
```

```
strcpy(str1, str2);  
strcat(dest1, src2);  
strcmp(dest1, src2);  
strcmpi(str1, str2);  
strlen(str);
```

```
#include<ctype.h>
```

```
tolower(ch);  
toupper(ch);
```



ฟังก์ชันที่สร้างขึ้นเอง (User Defined Function)

- เนื่องจากฟังก์ชันไลบรารีมาตรฐาน (Standard Library Function) ทั้งหมด เป็นฟังก์ชันที่มีเฉพาะการทำงานพื้นฐานต่างๆ เท่านั้น
- หากต้องการฟังก์ชันเฉพาะกิจ โปรแกรมเมอร์ต้องเขียนขึ้นมาเอง



1. มีการประกาศ function prototype ที่ต้นโปรแกรมเสมอ
จึงจะ เรียกใช้งาน function นั้นๆ ได้ (เป็นการบอก Compiler
ว่า คำสั่งดังกล่าวคือฟังก์ชัน ไม่ใช่ Syntax Error)
2. ต้องมีการเขียน Function ตามโครงสร้างที่ได้ประกาศไว้ใน
function prototype เท่านั้น



```
#include<file.h>
type function_name (type);
type variable
int main()
{
    type variable;
    statement-1;
    ...
    statement-n;
    return 0;
}
type function_name (type variable)
{
    statement-1;
    ...
    statement-n;
    return (var);
}
```

ฟรีโปรเซสเซอร์ไคเร็กทีฟ
ฟังก์ชันโปรโทไทป์
ตัวแปรชนิดโกบอล

ตัวแปรชนิดโลคอล
คำสั่ง

ฟังก์ชันหลัก

ฟังก์ชันย่อย

ส่วนหัวโปรแกรม

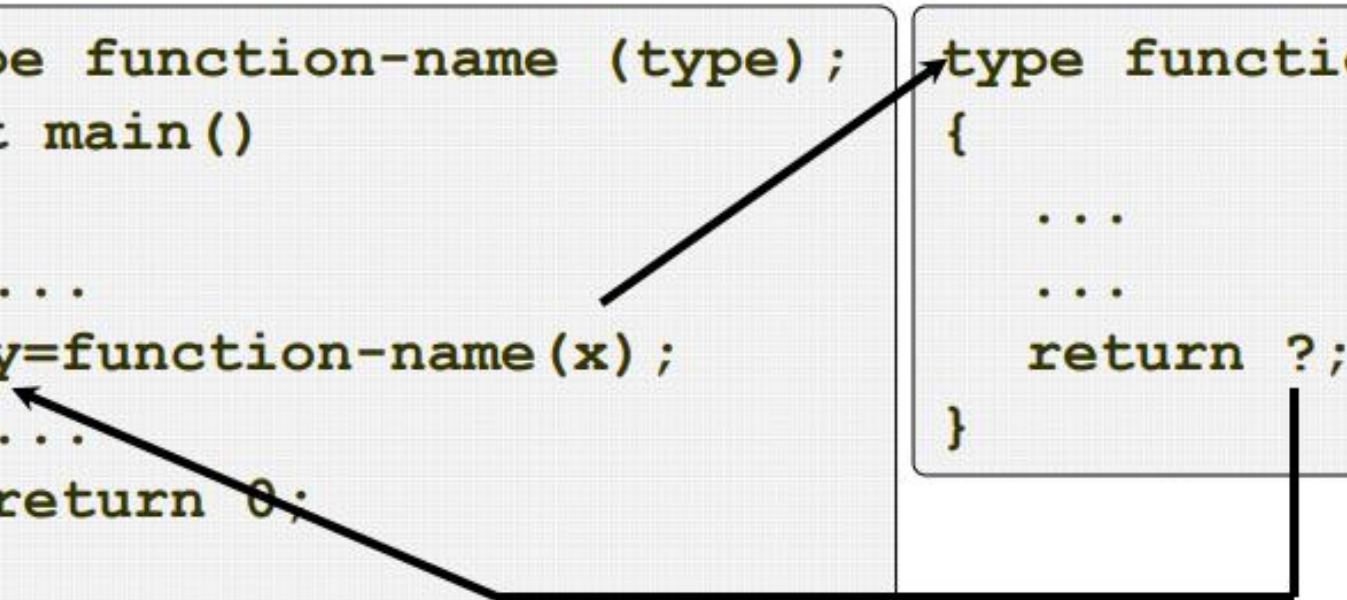
ส่วนตัวโปรแกรม



ฟังก์ชันที่สร้างขึ้นเอง (User Defined Function)

```
type function-name (type);  
int main()  
{  
    ...  
    y=function-name(x) ;  
    ...  
    return 0;  
}
```

```
type function-name (type)  
{  
    ...  
    ...  
    return ?;  
}
```





เป็นการประกาศใช้งานฟังก์ชันที่อยู่หลัง main()

```
type function_name (type-1 , type-2 , . . . , type-n) ;
```

Type คือ ชนิดของฟังก์ชัน ว่าฟังก์ชันที่ทำการสร้างจะส่งข้อมูลชนิดใดกลับ

Function name คือ ชื่อฟังก์ชันที่จะสร้างขึ้น

Type-n คือ ชนิดข้อมูลที่ส่งให้ฟังก์ชัน



- การเขียนโปรแกรมโดยมีฟังก์ชันที่สร้างขึ้นเองมี 2 รูปแบบ

1. สร้างฟังก์ชัน **ก่อน** ฟังก์ชันหลัก

- ฟังก์ชันหลักสามารถเรียกใช้ฟังก์ชันที่สร้างขึ้นได้

2. สร้างฟังก์ชัน **หลัง** ฟังก์ชันหลัก

- ต้องประกาศ Function Prototype ก่อน

เพื่อให้ฟังก์ชันหลักรู้ว่ามีการฟังก์ชันที่สร้างขึ้นได้



```
#include<file.h>
type variable
type function_name(type variable)
{
    statement-1;
    ...
    statement-n;
    return (var) ;
}
int main()
{
    type variable;
    statement-1;
    ...
    statement-n;
    return 0;
}
```

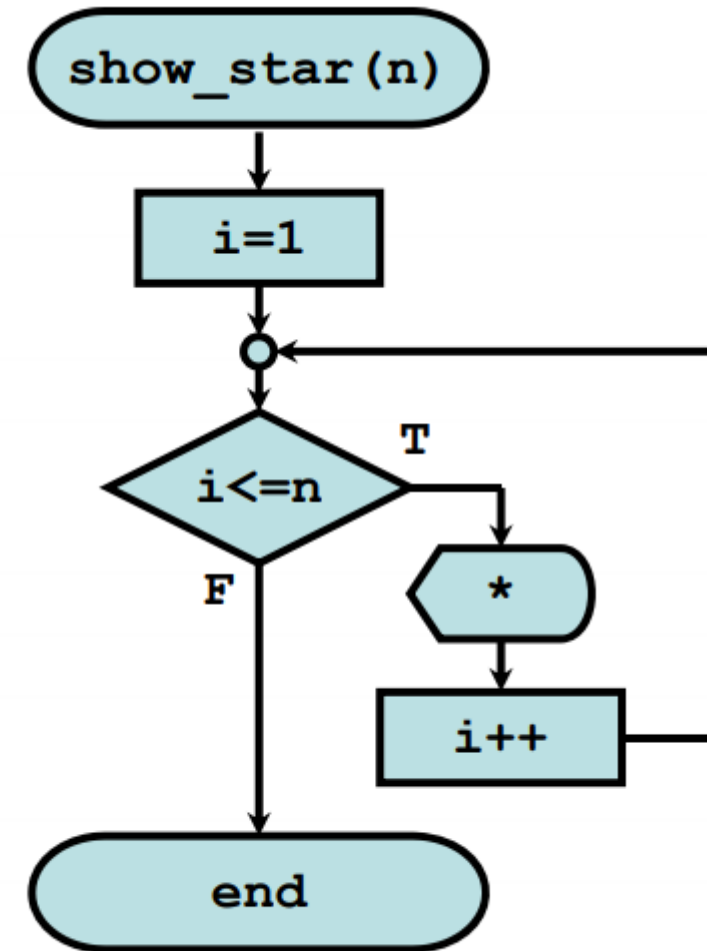
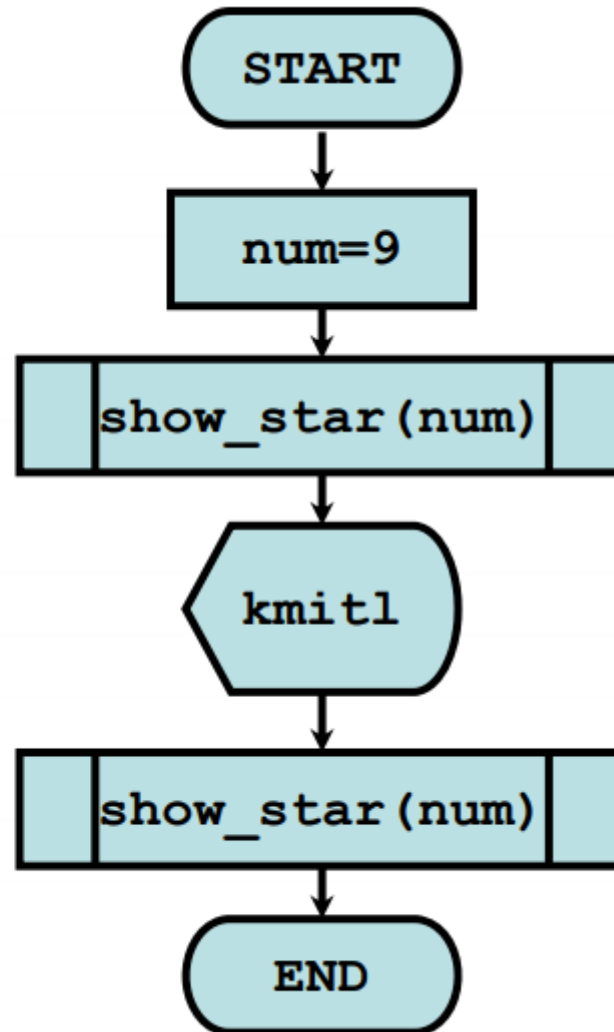


```
#include<file.h>
type function_name(type) ;
type variable
int main()
{
    type variable;
    statement-1;
    ...
    statement-n;
    return 0;
}
type function_name(type variable)
{
    statement-1;
    ...
    statement-n;
    return (var) ;
}
```



ตัวอย่างผังงาน & ผลลัพธ์

```
*****  
* kmitl *  
*****
```





```
#include<stdio.h>
#include<conio.h>
void show_star (int n)
{
    int i;
    for (i=1;i<=n;i++)
        putchar('*');
}
int main()
{
    int num=9;
    show_star(num);
    printf ("\n* kmitl *\n");
    show_star(num);
    return 0;
}
```

```
#include<stdio.h>
#include<conio.h>
void show_star (int);
int main()
{
    int num=9;
    show_star(num);
    printf ("\n* kmitl *\n");
    show_star(num);
    return 0;
}
void show_star (int n)
{
    int i;
    for (i=1;i<=n;i++)
        putchar('*');
}
```



- ฟังก์ชันที่ไม่มีการรับส่งค่า
- ฟังก์ชันที่มีการรับค่า แต่ไม่ส่งกลับ
- ฟังก์ชันที่มีการรับค่า และมีการส่งค่ากลับ
- ฟังก์ชันที่มีไม่มีการรับค่า แต่ มีการส่งค่ากลับ

**** สามารถแยกประเภทได้ โดยดูจาก Function Prototype****



```
#include<stdio.h>
void  PrintHello(void);
int main()
{
    PrintHello();
    printf("Hello,in function main.\n");
    PrintHello();
    return 0;
}
void  PrintHello(void)
{
    printf("Hello,in function PrintHello ..\n\n");
}
```



```
#include<stdio.h>
void PrintHello( int );
int main()
{
    int n;

    printf("input number of hello : ");
    scanf("%d",&n);
    PrintHello( n );
    return 0;
}

void PrintHello( int i )
{
    int count;
    for ( count=1; count<=i ; count++ )
        printf("%d HELLO\n",count);
}
```



```
#include<stdio.h>
float CircleArea( int );
int main()
{   int radius;
    printf("input radius : ");   scanf("%d",&radius);
    printf(" Circle area = %f\n",CircleArea(radius));
    return 0;
}

float CircleArea( int rad )
{   float answer=0;
    answer = 22.0/7.0*rad*rad ;
    return answer;
}
```



```
#include<stdio.h>
int RoundInput( void );
int main()
{ int i,round;
  round = RoundInput();
  for( i=1; i<=round;i++)   printf("hello #%d\n",i);
  return 0;
}
int RoundInput(void)
{ int answer;
  printf( "Please input number of hello : ");
  scanf( "%d", &answer );
  return  answer;
}
```



```
#include<stdio.h>
#include<math.h>
#define PI 3.14
float deg_rad(float);    /*Function Prototype*/
int main()
{
    float deg,rad;
    printf ("Enter Degree : ");
    scanf ("%f",&deg);
    rad = deg_rad(deg);    //printf ("%f->%f\n",deg,rad);
    printf ("sin(%.2f) = %.3f\n",deg,sin(rad));
    printf ("cos(%.2f) = %.3f\n",deg,cos(rad));
    printf ("tan(%.2f) = %.3f\n",deg,tan(rad));
    return 0;
}
float deg_rad(float num)
{
    float ans;
    ans = num * PI / 180;
    return(ans);
}
```

- 1.ฟังก์ชัน ชื่ออะไร
- 2.มีการรับค่าหรือไม่
ถ้ามีก็พารามิเตอร์
- 3.ฟังก์ชันรับค่าเป็นตัวแปรชนิดใด
- 4.ฟังก์ชันมีการส่งค่ากลับหรือไม่
ถ้ามีส่งค่าตัวแปรชนิดใดกลับ
- 5.ถ้าต้องการตัด /*Function Prototype*/
ต้องปรับเปลี่ยนโปรแกรมอย่างไร
- 6.โปรแกรมทำงานอะไร